1. **A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.**

Signed 8-bit 2's Complement Arithmetic

1. Problem Understanding

An 8-bit signed 2's-complement number uses the most significant bit as the sign indicator. A 0 in the MSB represents a non-negative value and a 1 represents a negative value. The representable range is:

```
-2^7 to (2^7 - 1) = -128 to +127
```

Both +45 and −23 lie inside this range. The processor must perform:

- +45 + (−23)
- +45 − (−23)
- Overflow detection for each operation.
- Interpretation of the final 8-bit bit patterns.

2. Method Selection

2's complement is the appropriate method because signed addition and subtraction can be performed with the same binary adder. Subtraction is converted to addition by taking the 2's complement of the subtrahend. This avoids requiring a separate subtractor for the basic arithmetic operation.

3. Binary Conversion of +45

Using positional weights 128, 64, 32, 16, 8, 4, 2 and 1:

$$45 = 32 + 8 + 4 + 1$$

$$45 = 00101101_2$$

Since +45 is positive, its 8-bit representation begins with 0.

4. Binary Conversion of −23

$$+23 = 00010111_2$$

Take the 1's complement:

$$00010111$$

$$11101000$$

Add 1 to obtain the 2's complement:

```
11101000
       + 1
-----------
11101001
```

Therefore, −23 = 11101001_2.

5. Addition: +45 + (−23)

```
  00101101
+ 11101001
-----------
1 00010110
```

The ninth carry is discarded because the processor retains 8 bits.

$$00010110_2 = 16 + 4 + 2 = 22$$

Therefore, the result is +22.

6. Subtraction: +45 − (−23)

Subtracting a negative number is equivalent to adding its positive magnitude:

```
45 − (−23) = 45 + 23
  00101101
+ 00010111
-----------
  01000100
```

$$01000100_2 = 64 + 4 = 68$$

Therefore, the result is +68.

7. Overflow Detection

For signed 2's-complement addition, overflow occurs if two positive operands produce a negative result or two negative operands produce a positive result. Another equivalent hardware condition is that the carry into the sign bit differs from the carry out of the sign bit.

Operation	Operand signs	Result	Overflow?
45 + (−23)	Positive + Negative	+22	No
45 − (−23) = 45 + 23	Positive + Positive	+68	No; 68 ≤ 127

For the second operation, although both operands are positive, +68 is still within the signed 8-bit range, so overflow does not occur.

8. Processor Interpretation

- The MSB is used to identify the sign.
- The same adder hardware can perform both addition and subtraction when 2's-complement control is used.
- An extra carry beyond the eighth bit is discarded in ordinary 2's-complement addition.
- Overflow is a signed-range condition, not simply the presence of a carry out.
- The final results are +22 and +68, both valid 8-bit signed values.

9. Final Answer

Item	Binary / Result
+45	00101101
−23	11101001
45 + (−23)	00010110 = +22
45 − (−23)	01000100 = +68
Overflow	No

2. **A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.**

4-bit Carry Look-Ahead Adder

1. Problem Understanding

A ripple carry adder calculates each stage after the previous stage has generated a carry. Therefore, a carry generated at the least significant bit may have to pass through all four stages. This creates a delay that increases as the word length increases.

A CLA reduces this dependency by deriving each carry directly from the input bits, propagate signals, generate signals and the initial carry.

2. Method Selection

The Carry Look-Ahead method is selected because the objective is specifically to reduce carry propagation delay. It uses combinational logic to calculate several carries in advance rather than waiting for a carry to ripple through every full-adder stage.

3. Generate and Propagate

For bit position i:

$$\textbf{Generate:} \quad G_i = A_i \cdot B_i$$

$$\textbf{Propagate:} \quad P_i = A_i \oplus B_i$$

$$\textbf{Carry:} \quad C_{i+1} = G_i + P_i C_i$$

A generate signal means the bit pair itself creates a carry. A propagate signal means an incoming carry can pass through that bit position.

4. Derivation of Carry Equations

$$C1 = G0 + P0C0$$

Substituting C1 into C2:

$$C2 = G1 + P1C1$$
$$= G1 + P1G0 + P1P0C0$$

Continuing the expansion:

$$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$$

$$C4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0C0$$

The sum bit is obtained from:

$$S_i = P_i \oplus C_i$$

5. Conceptual Working

1. Apply the two 4-bit operands A and B to the CLA.
2. Generate P and G for all four bit positions.
3. Use the look-ahead equations to calculate C1, C2, C3 and C4.
4. Compute each sum bit from P_i and C_i.
5. Produce the final 4-bit sum and carry-out

6. Ripple Carry versus CLA

Parameter	Ripple Carry Adder	4-bit CLA
Carry dependency	Stage-to-stage	Look-ahead equations
Carry calculation	Sequential	Parallel/combinational
Propagation delay	Higher	Lower
Hardware complexity	Low	Higher
Speed	Lower	Higher
Scalability	Delay increases strongly with bits	Can be organized in faster blocks
Processor relevance	Basic arithmetic	High-speed ALU design

7. Why CLA Improves Processor Speed

- It reduces the time spent waiting for carries to propagate through multiple full adders.
- The carry equations allow several carry values to be generated from the original inputs.
- Faster addition directly improves ALU operation time.
- CLA structures can be combined hierarchically to construct larger fast adders.
- The improvement is obtained at the cost of additional combinational hardware.

8. Interpretation

The key trade-off is speed versus hardware complexity. Ripple carry is simpler and uses less logic, but its worst-case delay increases with bit width. CLA uses more logic to obtain the carries earlier, making it attractive for high-performance processor arithmetic

3. **In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.**

Booth's Multiplication: (−6) × (+5)

1. Problem Understanding

The required multiplication is:

```
(−6) × (+5) = −30
```

Booth's algorithm operates on signed 2's-complement operands. For a 4-bit demonstration:

$-6 = 1010_2$

$+5 = 0101_2$

2. Registers Used

Register	Initial value	Purpose
M	1010	Multiplicand (−6)
Q	0101	Multiplier (+5)
A	0000	Accumulator / partial product
Q−1	0	Extra Booth bit used with Q0

3. Booth Decision Table

Q0 Q−1	Action
00	No addition/subtraction
01	A = A + M
10	A = A − M
11	No addition/subtraction

After the selected arithmetic operation, the combined A-Q-Q−1 register is shifted right arithmetically. The sign bit of A is replicated during the shift.

4. Cycle-by-Cycle Process

Initial:

```
A = 0000   Q = 0101   Q−1 = 0
```

Cycle 1: Q0Q−1 = 10 → A = A − M

```
0000 − 1010 = 0110
```

After arithmetic right shift: A = 0011, Q = 0010, Q−1 = 1

Cycle 2: Q0Q−1 = 01 → A = A + M

0011 + 1010 = 1101

After arithmetic right shift: A = 1110, Q = 1001, Q−1 = 0

Cycle 3: Q0Q−1 = 10 → A = A − M

1110 − 1010 = 0100

After arithmetic right shift: A = 0010, Q = 0100, Q−1 = 1

Cycle 4: Q0Q−1 = 01 → A = A + M

0010 + 1010 = 1100

After arithmetic right shift: A = 1110, Q = 0010

Final product is the combined A-Q register:

$$AQ = 11100010_2$$

5. Decimal Verification

$$11100010_2 \text{ is negative.}$$

1's complement = 00011101

Add 1 = 00011110 = 30

Therefore the signed result is −30, matching ordinary decimal multiplication.

6. Why Booth's Algorithm is Efficient

- It directly supports signed 2's-complement multiplication.
- It examines pairs Q0 and Q−1 rather than treating every multiplier bit independently.
- Runs of consecutive 1s can be replaced by a subtraction followed by a later addition.
- This can reduce the number of arithmetic operations compared with straightforward shift-and-add multiplication.
- Arithmetic right shifts preserve the sign during signed multiplication.

7. Comparison with Traditional Shift-and-Add

Feature	Traditional Shift-and-Add	Booth
Signed numbers	Needs sign handling	Naturally supports 2's complement
Decision basis	Multiplier bit	Q0 and Q−1 pair
Consecutive 1s	May cause many additions	Can reduce operations
Arithmetic shift	Not necessarily signed	Arithmetic right shift
Hardware application	Basic multiplier	Efficient signed multiplier

4. **A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.**

Restoring and Non-Restoring Division: 13 ÷ 3

1. Problem Understanding

Dividend = 13 = 1101_2

Divisor = 3 = 0011_2

Expected: 13 = (3 × 4) + 1

Therefore, the expected quotient is 4 and the expected remainder is 1.

2. Method Selection

Restoring division is selected to demonstrate the conventional binary division approach in which a negative trial remainder is restored. Non-restoring division is selected for comparison because it avoids performing that immediate restoration and therefore can reduce arithmetic operations.

3. Restoring Division – Algorithm

1. Initialize the accumulator A to zero and place the dividend in Q.
2. Shift the combined A-Q register left.
3. Subtract the divisor M from A.
4. If A becomes negative, restore A by adding M and set the new quotient bit to 0.
5. If A remains non-negative, keep A and set the new quotient bit to 1.
6. Repeat for the number of dividend bits.
7. The final Q contains the quotient and A contains the remainder.

4. Restoring Division – Intermediate State

Using A as a 5-bit partial remainder and Q as the 4-bit quotient register, the important decision is whether the trial subtraction A − M is negative or non-negative.

Iteration	Trial operation	Decision	Quotient bit
1	Shift then A − M	Negative → restore	0
2	Shift then A − M	Non-negative	1
3	Shift then A − M	Non-negative	1

4	Shift then A − M	Negative → restore	0

For a standard restoring implementation, the quotient bits are formed during the iterations; after the complete shift/subtract process, the arithmetic result is:

$$\text{Quotient} = 0100_2 = 4$$

$$\text{Remainder} = 0001_2 = 1$$

Verification: 3 × 4 + 1 = 13.

5. Non-Restoring Division – Algorithm

6. Initialize A = 0 and Q = dividend.
7. If the current A is non-negative, shift and subtract the divisor.
8. If the current A is negative, shift and add the divisor.
9. Set the quotient bit based on the sign of the resulting partial remainder.
10. Continue for all dividend bits.
11. If the final A is negative, add the divisor once to obtain the corrected remainder.

The main idea is that a negative partial remainder is allowed to remain negative temporarily. The next iteration compensates by adding the divisor instead of immediately restoring the previous value.

6. Final Non-Restoring Result

$$\text{Quotient} = 0100_2 = 4$$

$$\text{Remainder} = 0001_2 = 1$$

Verification:

$$13 = (3 \times 4) + 1$$

$$13 = 12 + 1$$

7. Comparison

Criterion	Restoring	Non-Restoring
Negative partial remainder	Immediately restored	Temporarily retained
Restoration operation	Can occur repeatedly	Avoided during intermediate stages
Number of arithmetic operations	Generally higher	Generally lower
Control logic	Straightforward	More conditional
Efficiency	Lower for repeated restoration	Generally better
Processor relevance	Useful for understanding/basic units	Attractive for efficient arithmetic units

8. Interpretation

Both algorithms implement the same mathematical division and produce quotient 4 and remainder 1. The major architectural advantage of non-restoring division is that it avoids an explicit restoration after every negative trial remainder. This can reduce work in the arithmetic unit.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

IEEE 754 Representation and Floating-Point Addition

1. Problem Understanding

IEEE 754 stores a floating-point number using a sign field, a biased exponent and a fraction/significand field. The same mathematical value has different bit-field sizes in single and double precision.

Format	Sign	Exponent	Fraction	Total
Single precision	1 bit	8 bits	23 bits	32 bits
Double precision	1 bit	11 bits	52 bits	64 bits

2. Method Selection

The IEEE 754 conversion procedure is selected: convert the decimal value to binary, normalize it, determine the sign, add the appropriate exponent bias, and place the fractional bits into the fraction field.

3. Convert −13.25 to Binary

Integer part:

$$13 = 1101_2$$

Fractional part:

$$0.25 = 0.01_2$$

Therefore:

$$13.25 = 1101.01_2$$

$$-13.25 = -1101.01_2$$

4. Normalize

$$1101.01_2 = 1.10101_2 \times 2^3$$

Therefore:

- Sign bit = 1 because the number is negative.
- Actual exponent = 3.

- Significand begins with 1.10101.
- The leading 1 is implicit in normalized IEEE 754 representation.

5. Single Precision

Single precision uses a bias of 127.

Biased exponent = 3 + 127 = 130

$130 = 10000010_2$

Fraction field:

10101000000000000000000

Complete field layout:

Sign | Exponent | Fraction

1 | 10000010 | 10101000000000000000000

11000001010101000000000000000000

Thus the 32-bit IEEE 754 representation is:

11000001010101000000000000000000

6. Double Precision

Double precision uses a bias of 1023.

Biased exponent = 3 + 1023 = 1026

$1026 = 10000000010_2$

Fraction field:

1010100000000000000000000000000000000000000000000000

Complete layout:

1 | 10000000010 | 1010100000000000000000000000000000000000000000000000000000

7. Floating-Point Addition – Detailed Procedure

Floating-point addition is not simply integer addition of the stored bit patterns. The processor must first make the two operands comparable.

1. Unpack the operands into sign, exponent and significand.
2. Compare the two exponents.
3. Shift the significand of the operand with the smaller exponent so both operands have the same exponent.
4. Perform significand addition or subtraction according to the operand signs.
5. Normalize the resulting significand.

6. Adjust the exponent after normalization.
7. Round the significand to the available number of fraction bits.
8. Check for overflow, underflow and special values where applicable.

8. Example of Exponent Alignment

Suppose two normalized values have the form:

$$X = 1.10100 \times 2^{4}$$

$$Y = 1.01000 \times 2^{2}$$

The exponent of Y is smaller. Shift its significand right by two places:

$$Y = 0.01010 \times 2^{4}$$

Now both operands have exponent 4 and their significands can be combined. The exact operation depends on their signs.

9. Normalization

After significand arithmetic, the result may not be in normalized form. For example, if an addition produces 10.011×2^{4}, shift the binary point one position to the left and increase the exponent:

$$10.011 \times 2^{4} = 1.0011 \times 2^{5}$$

If cancellation produces a value such as 0.00101×2^{4}, the significand can be shifted left and the exponent reduced until the normalized form is restored.

10. Rounding and Precision Loss

IEEE 754 formats have finite fraction fields. If an exact result needs more bits than the format can store, rounding is required. The stored value can therefore differ slightly from the exact mathematical value.

- Single precision stores 23 explicit fraction bits.
- Double precision stores 52 explicit fraction bits.
- More fraction bits generally provide greater precision.
- Rounding can occur after alignment and normalization.
- Repeated floating-point operations can accumulate small errors.

11. Common Floating-Point Issues

Issue	Meaning
Rounding error	The exact result cannot fit into the available fraction field.

Precision loss	Significant digits may be lost during arithmetic.
Overflow	The exponent exceeds the representable range.
Underflow	The magnitude becomes too small for normal representation.
Cancellation	Subtracting nearly equal numbers can remove significant digits.
Exponent alignment	A smaller-exponent operand may require right shifting before addition.

12. Single versus Double Precision

Feature	**Single Precision**	**Double Precision**
Total bits	32	64
Exponent bits	8	11
Fraction bits	23	52
Bias	127	1023
Precision	Lower	Higher
Storage	Less	More
Typical use	Graphics, embedded and general numerical work	Scientific and high-precision computation

13. Interpretation

IEEE 754 allows processors to represent a very wide range of real-valued quantities in a standardized format. Double precision offers a larger fraction field and exponent field, which generally improves numerical precision and range at the cost of additional storage and hardware resources.